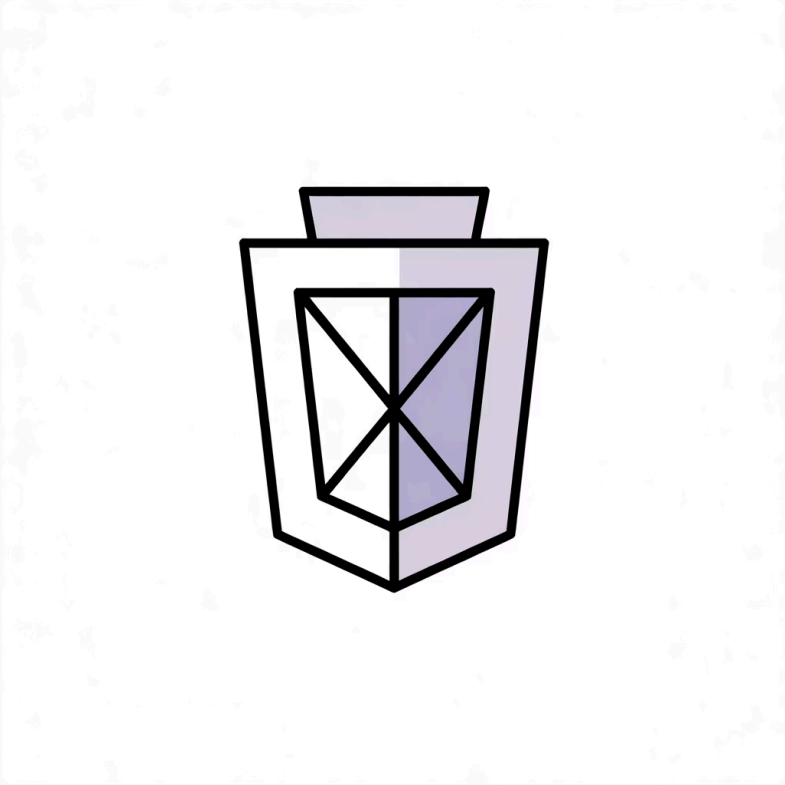
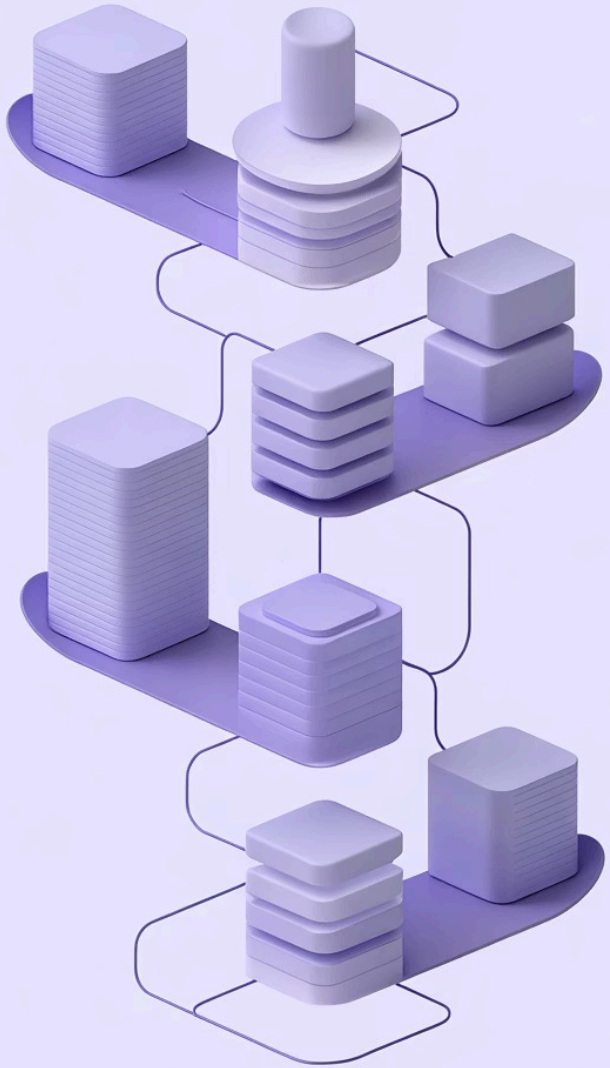
JavaScript - Jour 3

Découverte et manipulation du DOM



Objectifs pédagogiques de la journée

- Introduction au DOM
- Sélection d'éléments
- Modification du contenu
- Modifications des styles et attributs
- Insertion et suppression d'éléments
- Atelier de mise en pratique



MODULE

Introduction au DOM

EXPLICATIONS

Qu'est-ce que le DOM ?

Le DOM (Document Object Model) est une représentation en mémoire de la structure HTML de votre page web. C'est une interface de programmation qui permet à JavaScript d'accéder et de manipuler le contenu, la structure et le style d'un document HTML.

EXPLICATIONS

La structure arborescente du DOM

Le DOM organise tous les éléments HTML de la page en une structure d'arbre :

```
document
  └─── html
     ├─── head
     │   ├─── title
     │   └─── meta
     └─── body
        ├─── header
        │   └─── h1
        ├─── main
        │   ├─── section
        │   │   ├─── h2
        │   │   └─── p
        │   └─── section
        └─── footer
```

EXPLICATIONS

Le vocabulaire du DOM

Chaque élément HTML de la page est un "nœud" dans la structure arborescente du DOM. Ces nœuds entretiennent des relations spécifiques, souvent décrites avec un vocabulaire familial pour faciliter la compréhension :

Parent

Le nœud qui se trouve directement au-dessus d'un autre dans la hiérarchie.

Enfant (Child)

Les nœuds qui sont directement contenus par un nœud parent.

Frères/Sœurs (Siblings)

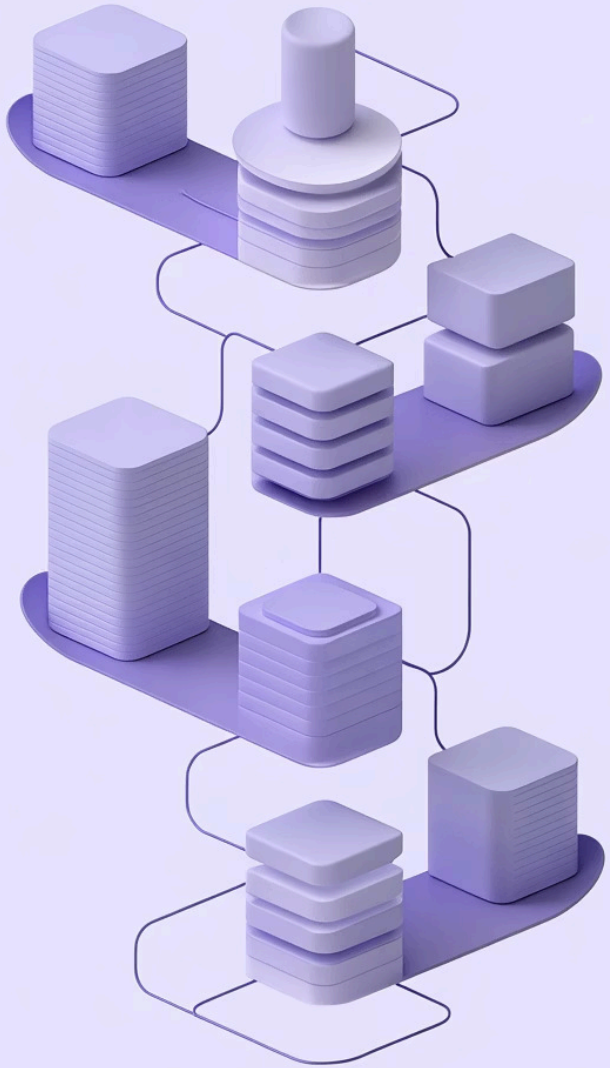
Les nœuds qui partagent le même parent et sont au même niveau.

EXPLICATIONS

L'objet document

L'objet `document` est le point d'entrée vers le DOM. Il représente toute la page web et offre de nombreuses méthodes pour sélectionner et manipuler les éléments.

```
console.log(document);      // Affiche l'objet Document en entier (la page web)
console.log(document.title); // Accède au titre de la page ("Titre de ma page")
console.log(document.URL);   // Récupère l'URL complète de la page actuelle
```



MODULE

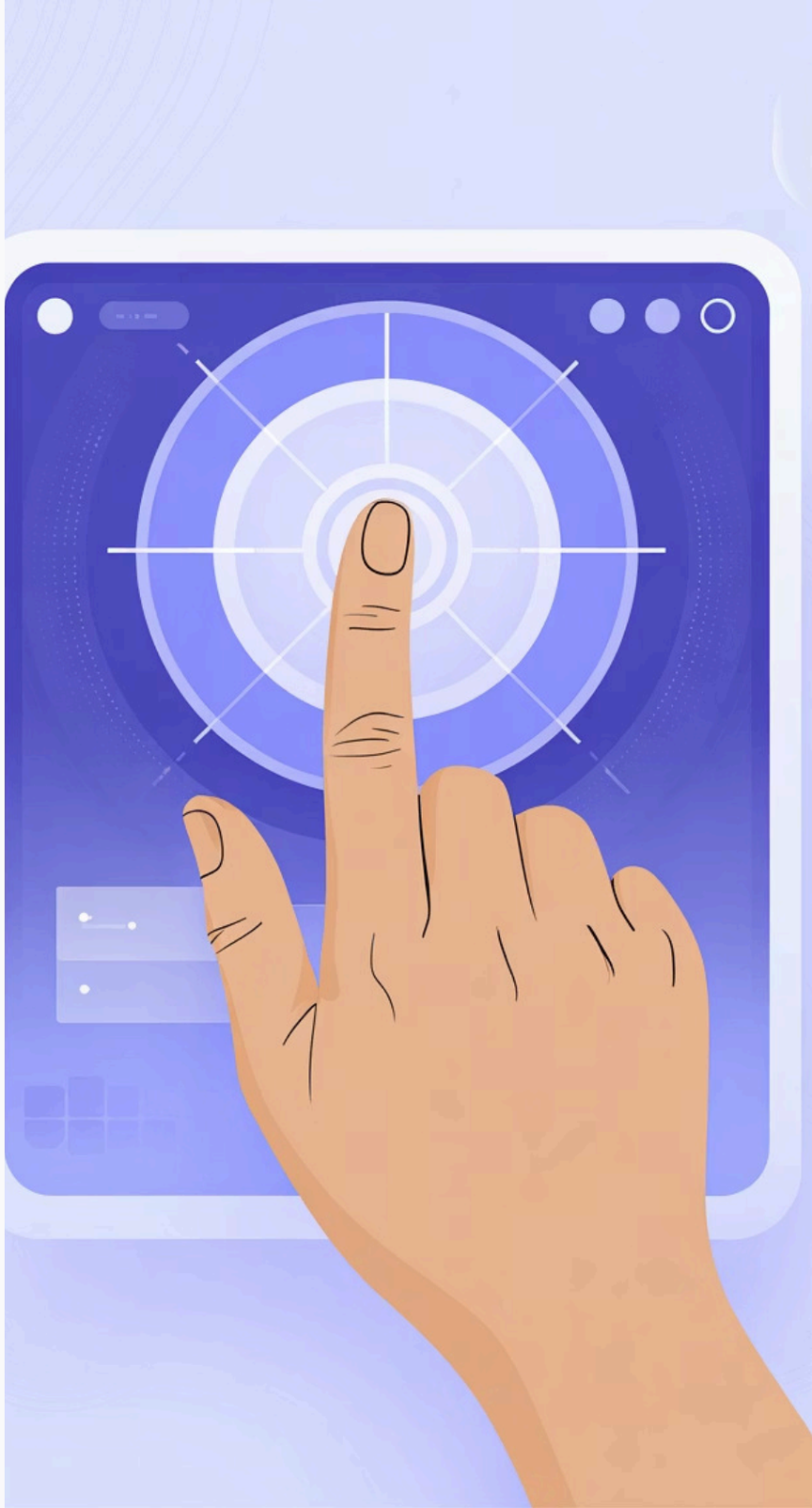
Sélection d'éléments

EXPLICATIONS

Concepts théoriques

Pour manipuler les éléments HTML, il faut d'abord les sélectionner. JavaScript offre plusieurs méthodes pour cibler précisément les éléments dont vous avez besoin.

- getElementById()
- querySelector()
- querySelectorAll()
- getElementsByClassName()
- getElementsByTagName()



EXEMPLE

getElementById()

Sélectionne **UN** élément par son attribut `id`. Retourne l'élément ou `null` si non trouvé.

```
const title = document.getElementById('main-title');
console.log(title); // <h1 id="main-title">Bienvenue</h1>
```

Points importants

- Les IDs doivent être uniques dans une page.
- Méthode la plus rapide.
- Retourne directement l'élément (pas une collection).

EXEMPLE

querySelector()

Sélectionne **LE PREMIER** élément qui correspond au sélecteur CSS fourni.

```
// Par ID
const title = document.querySelector('#main-title');

// Par classe
const firstDescription = document.querySelector('.description');

// Par balise
const firstParagraph = document.querySelector('p');

// Sélecteurs complexes
const firstLinkInNav = document.querySelector('nav a');
const firstListItem = document.querySelector('ul.menu > li');
```

Avantages

- Syntaxe familière
- Très flexible
- Retourne directement l'élément

EXEMPLE

querySelectorAll()

Sélectionne **TOUS** les éléments qui correspondent au sélecteur CSS fourni. Retourne une **NodeList**.

```
// Tous les paragraphes
const allParagraphs = document.querySelectorAll('p');

// Tous les éléments avec la classe 'description'
const allDescriptions = document.querySelectorAll('.description');

// Tous les liens dans la navigation
const navLinks = document.querySelectorAll('nav a');

// Parcourir les résultats
navLinks.forEach((link) => console.log(link.textContent));
```

Points importants

- **Retourne une NodeList (collection statique).**
- **Peut être parcourue avec** `forEach()`**.**
- **Idéale pour cibler et manipuler plusieurs éléments.**

EXEMPLE

getElementsByClassName()

Sélectionne **TOUS** les éléments ayant une classe spécifique. Retourne une **HTMLCollection** "live".

```
const descriptions = document.getElementsByClassName('description');
console.log(descriptions.length); // Nombre d'éléments

// Conversion en Array pour utiliser les méthodes de tableaux
const descriptionsArray = Array.from(descriptions);
descriptionsArray.forEach((description) => console.log(description));
```

Points importants

1

Retourne une **HTMLCollection** "live" (met à jour dynamiquement la liste si le DOM change).

2

Nécessite une conversion en tableau (`Array.from()` ou spread operator) pour utiliser des méthodes comme `forEach()`.

3

Idéale pour cibler des groupes d'éléments basés sur une classe CSS commune.

EXEMPLE

getElementsByTagName()

Sélectionne **TOUS** les éléments d'un type de balise spécifique. Retourne une **HTMLCollection** "live".

```
const allParagraphs = document.getElementsByTagName('p');
const allDivs = document.getElementsByTagName('div');
const allLinks = document.getElementsByTagName('a');

// Parcourir avec une boucle for classique
for (let i = 0; i < allParagraphs.length; i++) {
  console.log(allParagraphs[i]);
}
```

Points importants

1. Retourne une **HTMLCollection** "live" (met à jour dynamiquement la liste).

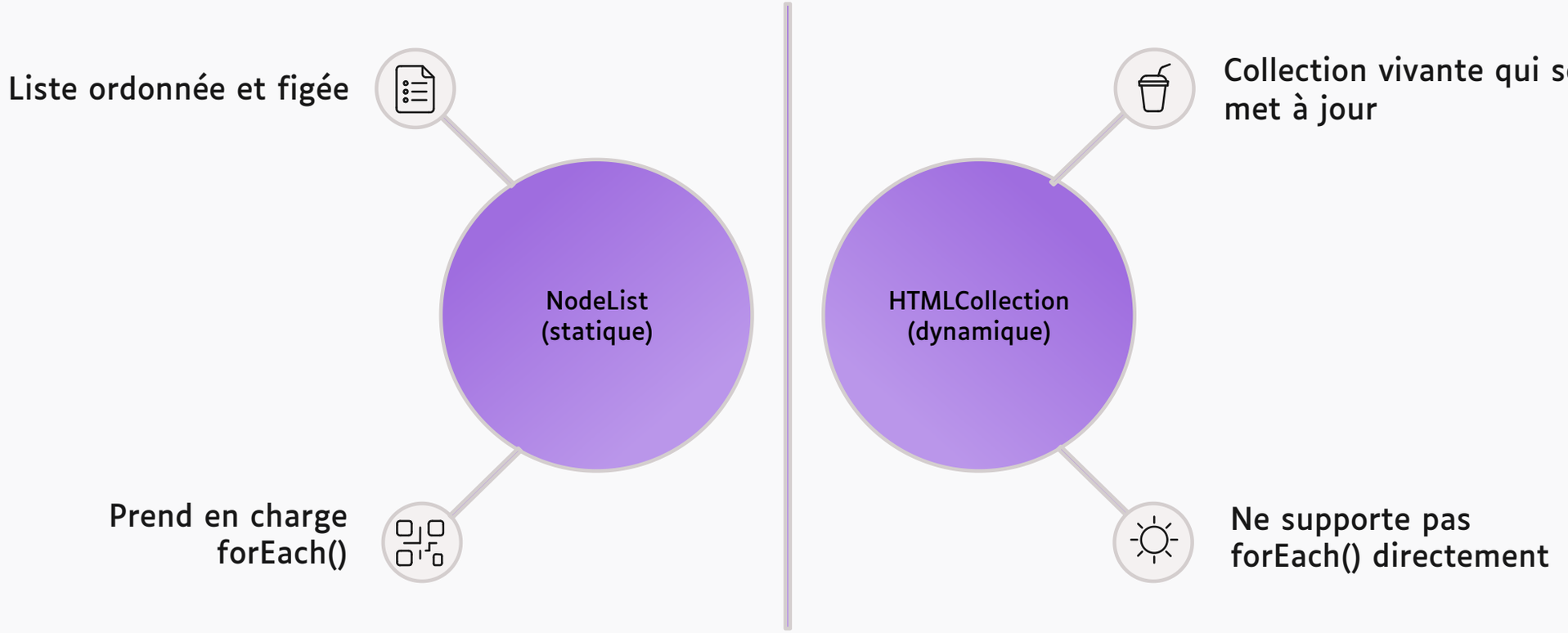
2. Moins spécifique, utile pour des types de balises génériques.

3. Nécessite une conversion en tableau pour les méthodes comme `forEach()`.

EXPLICATIONS

NodeList vs HTMLCollection

Comprendre les différences fondamentales entre ces deux collections est crucial pour manipuler le DOM efficacement en JavaScript.



EXPLICATIONS

Quelle méthode de sélection choisir ?

GETELEMENTBYID()

Pour un élément unique et spécifique

QUERYSELECTOR()

Pour le premier élément correspondant à un sélecteur CSS

QUERYSELECTORALL()

Pour tous les éléments correspondant à un sélecteur CSS

GETELEMENTSBYCLASSNAME()

GETELEMENTSBYTAGNAME()

Pour une compatibilité maximale (anciens navigateurs)

Recommandation moderne : Privilégiez `getElementById()`, `querySelector()` et `querySelectorAll()` pour leur simplicité, leur flexibilité et leur puissance.

MISE EN PRATIQUE

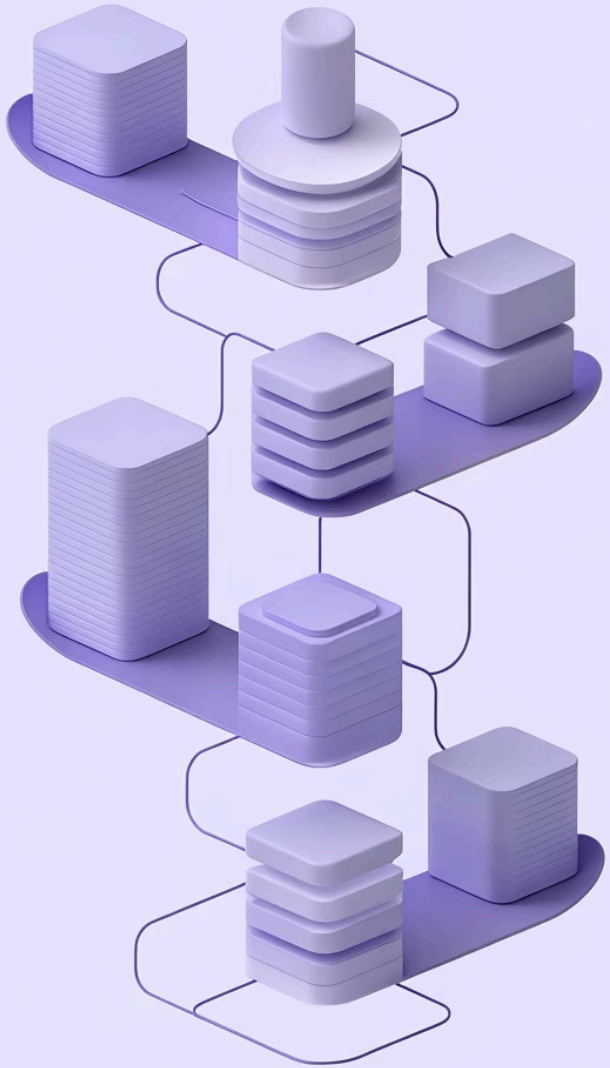
Mise en pratique

À partir d'un fichier HTML fourni, réalisez un script JavaScript qui s'exécute après le chargement de la page et qui doit :

- Récupérer tous les liens (balise `<a>`).
- Récupérer tous les services affichés (par exemple, des éléments avec une classe `.service`).
- Récupérer le témoignage de Pierre (par exemple, un élément avec l'ID `#temoignage-pierre`).

Mise en pratique - HTML

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript Jour 3 - DOM</title>
</head>
<body>
    <div id="container">
        <header id="main-header" class="header">
            <h1 class="title main-title">Bienvenue sur notre site</h1>
            <nav class="navigation">
                <ul class="menu">
                    <li class="menu-item"><a href="#home">Accueil</a></li>
                    <li class="menu-item"><a href="#services">Services</a></li>
                    <li class="menu-item"><a href="#about">À propos</a></li>
                    <li class="menu-item"><a href="#contact">Contact</a></li>
                </ul>
            </nav>
        </header>

        <main id="content" class="main-content">
            <section class="hero">
                <h2 class="section-title">Section héro</h2>
                <p class="description">Découvrez nos services exceptionnels.</p>
                <button class="btn btn-primary">En savoir plus</button>
            </section>

            <section class="services">
                <h2 class="section-title">Nos services</h2>
                <div class="service-card" data-service="web">
                    <h3>Développement Web</h3>
                    <p class="description">Sites web modernes et responsive.</p>
                </div>
                <div class="service-card" data-service="mobile">
                    <h3>Applications Mobile</h3>
                    <p class="description">Applications iOS et Android.</p>
                </div>
                <div class="service-card" data-service="design">
                    <h3>Design UI/UX</h3>
                    <p class="description">Interfaces élégantes et intuitives.</p>
                </div>
            </section>

            <section class="testimonials">
                <h2 class="section-title">Témoignages</h2>
                <blockquote class="testimonial" data-author="Marie">
                    <p>"Excellent travail, très professionnel !"</p>
                </blockquote>
                <blockquote class="testimonial" data-author="Pierre">
                    <p>"Je recommande vivement leurs services."</p>
                </blockquote>
            </section>
        </main>

        <footer id="main-footer" class="footer">
            <p class="copyright">&copy; 2026 Coda</p>
            <div class="social-links">
                <a href="#" class="social-link">Facebook</a>
                <a href="#" class="social-link">Twitter</a>
                <a href="#" class="social-link">LinkedIn</a>
            </div>
        </footer>
    </div>

    <script src="script.js"></script>
</body>
</html>
```



MODULE

Modification du contenu

EXPLICATIONS

Les trois propriétés

Une fois un élément sélectionné, nous pouvons modifier son contenu de plusieurs manières à l'aide de ces trois propriétés JavaScript disponible sur un élément individuel.

- `textContent`
- `innerHTML`
- `innerText`

EXPLICATIONS

textContent

Récupère ou définit le contenu textuel d'un élément et de tous ses descendants.

Caractéristiques

- **Retourne TOUT le texte, même celui des éléments cachés (`display: none`).**
- **Ignore les balises HTML, les traitant comme du texte brut.**
- **Méthode rapide et sûre (prévient l'injection de code malveillant).**

```
const title = document.querySelector('h1');

// Lire le contenu textuel
console.log(title.textContent); // Exemple : "Bienvenue sur notre site"

// Modifier le contenu textuel
title.textContent = "Nouveau titre de la page";

// Le HTML est échappé automatiquement et affiché comme du texte
title.textContent = "<strong>Titre en gras</strong>"; // Ne sera pas interprété en HTML mais en texte
```

EXPLICATIONS

innerHTML

Récupère ou définit le contenu HTML d'un élément, permettant d'intégrer des balises et des structures complexes.

Caractéristiques

- Interprète les balises HTML.
- Permet d'insérer du HTML dynamiquement.
- Risque de faille XSS

```
const section = document.querySelector('.main-content');

// Lire le HTML
console.log(section.innerHTML);

// Modifier avec du HTML
section.innerHTML = `
  <h2>Nouveau contenu</h2>
  <p>Avec du <strong>HTML</strong> formaté</p>
`;

// Ajouter du contenu (attention, remplace tout)
section.innerHTML += '<p>Paragraphe supplémentaire</p>';
```

EXPLICATIONS

innerText

Similaire à `textContent` mais respecte le style CSS.

Caractéristiques

Respecte le style CSS (n'affiche pas le texte masqué par `display: none` ou `visibility: hidden`).

Prend en compte le style de l'élément (par exemple, les retours à la ligne et les espaces).

Plus lent que `textContent`

Déclenche un "reflow" (recalcul de la mise en page)

```
const element = document.querySelector('.description');

// Si l'élément contient du texte caché (display: none),
// innerText ne le retournera pas, contrairement à textContent
console.log(element.innerText);
```

EXPLICATIONS

Quelle propriété utiliser ?

`textContent`

Pour modifier du texte simple. Recommandé pour sa performance et sa sécurité, car il traite le contenu comme du texte brut.

`innerText`

Rarement nécessaire. Utilisez `textContent` à la place, car `innerText` est plus lent et déclenche un recalcul de la mise en page.

`innerHTML`

Pour insérer du HTML dynamiquement. À utiliser avec prudence en raison des potentielles failles de sécurité (XSS).

⚠ Pour le moment on s'autorise l'utilisation de `innerHTML`, mais par la suite on utilisera une autre solution plus sécurisée.

MISE EN PRATIQUE

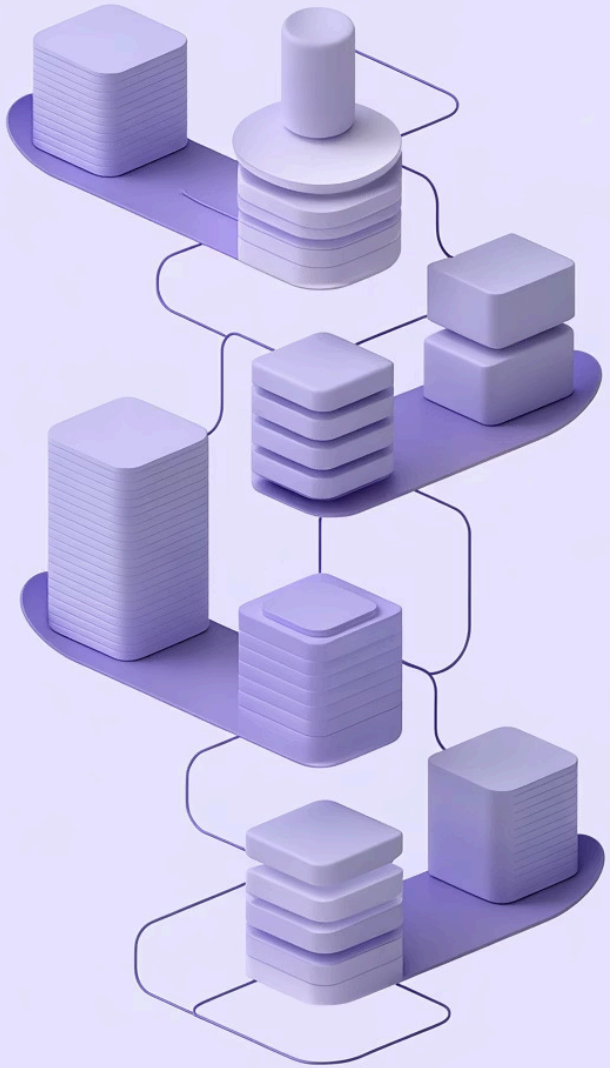
Interaction utilisateur et modification du DOM

Créez un programme JavaScript qui recueille les informations de l'utilisateur et les intègre dynamiquement dans le contenu de la page web.

Objectif

L'utilisateur devra être invité à fournir son prénom et son âge. Ces données devront ensuite être affichées à des endroits spécifiques dans le HTML.

Pensez à préparer votre fichier HTML en y ajoutant une balise `<div id="user-infos"></div>` qui contiendra toutes les informations de l'utilisateur.



MODULE

Modification des styles

EXPLICATIONS

Modification des styles

JavaScript permet de modifier l'apparence des éléments de deux manières principales : en modifiant directement les styles inline ou en manipulant les classes CSS.

Propriété style

Modifie directement les styles CSS de l'élément en ligne.

classList (Recommandé)

Ajoute, supprime ou bascule des classes CSS prédéfinies.

EXPLICATIONS

Propriété style

Permet de modifier les styles CSS directement sur un élément (styles inline).

Syntaxe et conventions

- **Les propriétés CSS avec tirets deviennent camelCase :** `background-color` → `backgroundColor`
- **Toutes les valeurs de style doivent être des chaînes de caractères (exemple :** `"20px"`, `"red"`, `"1.5em"`**).**

```
const monElement = document.getElementById('mon-element');

// Changer la couleur du texte en rouge
monElement.style.color = 'red';

// Changer la taille de police
monElement.style.fontSize = '20px';

// Changer la couleur d'arrière-plan
monElement.style.backgroundColor = '#ADD8E6';

// Lire un style
console.log(monElement.style.color); // "red"
```

EXPLICATIONS

Styles calculés : Lire les CSS réels

La propriété `element.style.property` est utile pour définir des styles inline, mais elle ne permet de récupérer que ces styles inline. Elle n'accède pas aux styles définis dans les feuilles de style CSS externes, les balises `<style>` ou les styles hérités.

Limitation

`element.style.property` **ne retourne que les styles définis directement sur l'élément (styles inline).**

Solution

Pour lire les styles CSS réellement appliqués à un élément (styles calculés), utilisez la méthode `window.getComputedStyle()`**.**

```
const monElement = document.getElementById('mon-element');

// Lire un style défini dans une feuille de style CSS
const computedStyles = window.getComputedStyle(monElement);
console.log(computedStyles.color);          // Ex: "rgb(0, 0, 0)"
console.log(computedStyles.fontSize);       // Ex: "16px"
console.log(computedStyles.backgroundColor); // Ex: "rgb(255, 255, 255)"

// Note : computedStyles retourne la valeur finale et absolue du style.
```

EXPLICATIONS

classList

La propriété `classList` offre un moyen robuste et recommandé de manipuler les classes CSS d'un élément, assurant ainsi une meilleure séparation des préoccupations entre le JavaScript (comportement) et le CSS (style).

add(...classes)

Ajoute une ou plusieurs classes à l'élément.

remove(...classes)

Supprime une ou plusieurs classes de l'élément.

toggle(class)

Ajoute la classe si elle est absente, la supprime si elle est présente.

contains(class)

Vérifie si une classe spécifique est présente, retournant `true` ou `false`.

replace(oldClass, newClass)

Remplace une classe existante par une nouvelle.

EXPLICATIONS

classList.add()

La méthode `add()` permet d'ajouter une ou plusieurs classes CSS à un élément. Si une classe est déjà présente, elle ne sera pas ajoutée une seconde fois.

Utilisation

- Prend un ou plusieurs arguments, chaque argument étant une chaîne de caractères représentant le nom d'une classe.
- Idéal pour appliquer des styles conditionnels ou déclenchés par des interactions utilisateur.

```
const monBouton = document.getElementById('monBouton');

// Ajouter une seule classe
monBouton.classList.add('active');

// Ajouter plusieurs classes en même temps
monBouton.classList.add('highlight', 'bordered');
```

EXPLICATIONS

classList.remove()

La méthode `remove()` permet de supprimer une ou plusieurs classes CSS d'un élément. Si une classe spécifiée n'est pas présente, aucune erreur n'est générée et l'opération est simplement ignorée.

Utilisation

- Prend un ou plusieurs arguments, chaque argument étant une chaîne de caractères représentant le nom d'une classe.
- Idéal pour réinitialiser des états visuels ou modifier l'apparence en réponse à des actions utilisateur, comme la désactivation d'un bouton.

```
const monBouton = document.getElementById('mon-bouton');

// Supprimer une seule classe
monBouton.classList.remove('active');

// Supprimer plusieurs classes en même temps
monBouton.classList.remove('highlight', 'bordered');
```

EXPLICATIONS

classList.toggle()

La méthode `toggle()` offre un moyen extrêmement pratique de basculer la présence d'une classe sur un élément. Si la classe existe, elle est supprimée ; sinon, elle est ajoutée.

Utilisation

- Prend un argument obligatoire : le nom de la classe à basculer.
- Un second argument optionnel (booléen) peut forcer l'ajout (`true`) ou la suppression (`false`) de la classe, ignorant sa présence actuelle.
- C'est la solution idéale pour implémenter des comportements interactifs simples comme l'ouverture/fermeture de menus, l'activation/désactivation d'éléments, ou l'application de thèmes.

```
const monElement = document.getElementById('mon-element');

// Bascule automatiquement sur l'ajout/suppression de la classe
monElement.classList.toggle('active');

// Forcer l'ajout de la classe 'active'
monElement.classList.toggle('active', true);

// Forcer la suppression de la classe 'active'
monElement.classList.toggle('active', false);
```

EXPLICATIONS

classList.contains()

La méthode `contains()` permet de vérifier si un élément possède une classe CSS spécifique. Elle retourne `true` si la classe est présente, et `false` sinon.

Utilisation

- Prend un seul argument obligatoire : le nom de la classe à vérifier.
- Extrêmement utile pour la logique conditionnelle en JavaScript, par exemple pour savoir si un élément est dans un état particulier (actif, caché, sélectionné, etc.) avant d'effectuer une action.

```
const monElement = document.getElementById('mon-element');

if (monElement.classList.contains('highlight')) {
  console.log("L'élément est mis en évidence.");
} else {
  console.log("L'élément n'est pas mis en évidence.");
}
```

EXPLICATIONS

classList.replace()

La méthode `replace()` permet de remplacer une classe CSS existante par une nouvelle sur un élément. Si la classe à remplacer n'existe pas, l'opération échoue silencieusement sans provoquer d'erreur.

Utilisation

- Prend deux arguments obligatoires : le nom de la classe à remplacer (`oldClass`) et le nom de la nouvelle classe (`newClass`).
- Particulièrement utile pour gérer des transitions d'état complexes où une classe spécifique doit être retirée et une autre appliquée simultanément.

```
const monElement = document.getElementById('mon-element');

// Supposons que 'mon-element' ait la classe 'old-style'
monElement.classList.replace('old-style', 'new-style');

// Si 'mon-element' n'avait pas 'old-style', rien ne se passe.
```

⚖️ EXPLICATIONS

style vs classList : Quel choix faire ?

Critère	Propriété `style`	Objet `classList`
Utilisation	Styles inline directs	Manipulation de classes CSS
Maintenabilité	❌ Mélange JS et style	✅ Séparation claire
Performance	⚠️ Moyenne (reflow)	✅ Bonne (classes précalculées)
Réutilisabilité	❌ Difficile	✅ Excellente
Cas d'usage	Styles dynamiques calculés	Changements d'état prédéfinis

EXEMPLE

Exemple d'utilisation de classList

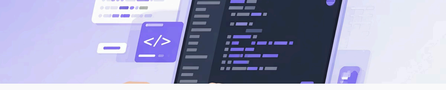
```
const body = document.body;
const themeIndicator = document.querySelector("#theme-indicator");

function onChangeTheme() {
  body.classList.toggle("dark-theme");
  const isDarkTheme = body.classList.contains("dark-theme");

  themeIndicator.textContent = isDarkTheme ? "Thème sombre" : "Thème clair";
  themeIndicator.classList.toggle("button-yellow", !isDarkTheme);
});
```

Qu'est-ce que c'est que cette syntaxe `isDarkTheme ? "texte1" : "texte2"`**, et comment ça fonctionne ?**

EXEMPLE

Exemple d'utilisation combinée

```
function createProgressBar(percentage) {
  const bar = document.querySelector(".progress-bar");

  // Utilisation de style pour la largeur dynamique
  bar.style.width = `${percentage}%`;

  // Utilisation de classList pour la gestion des couleurs conditionnelles
  bar.classList.remove("bar-red", "bar-orange", "bar-green");

  // Couleur selon le pourcentage
  if (percentage < 33) {
    bar.classList.add("bar-red");
  } else if (percentage < 66) {
    bar.classList.add("bar-orange");
  } else {
    bar.classList.add("bar-green");
  }
}
```



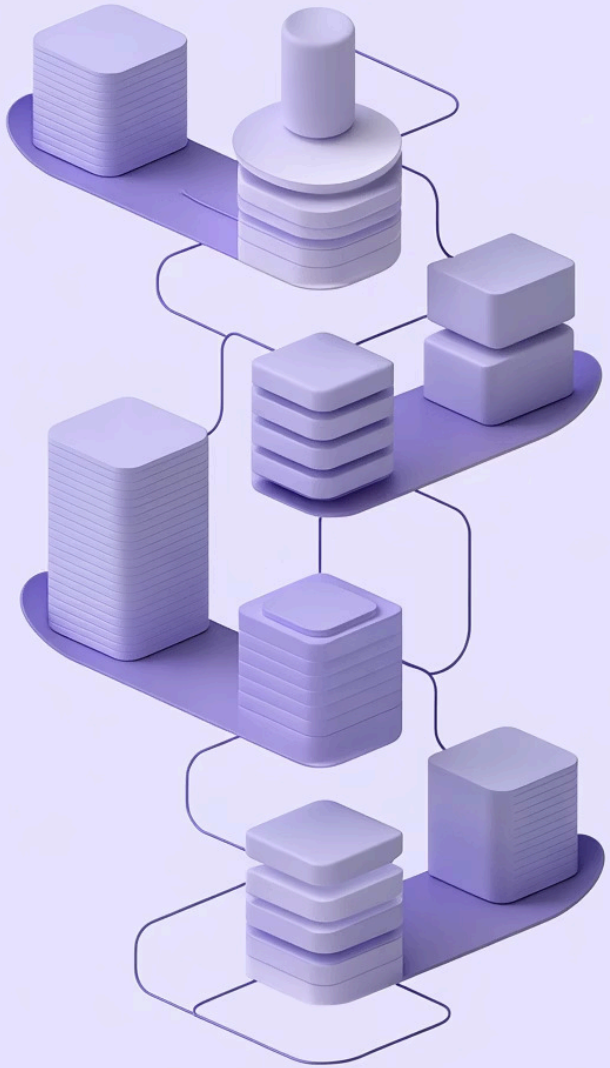
MISE EN PRATIQUE

Mise en pratique

À partir du code HTML/CSS de base fourni, développez les fonctions JavaScript listées dans les attributs `onclick` **des boutons. Ces fonctions manipuleront les classes CSS et les styles directs des éléments.**

▼ **HTML de base**

```
<!DOCTYPE html>
<html lang="fr">
  <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Styliste Dynamique</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: Arial, sans-serif;
      padding: 20px;
      transition: background-color 0.3s, color 0.3s;
    }

    .container {
      max-width: 1200px;
      margin: 0 auto;
    }

    .controls {
      background-color: #f5f5f5;
      padding: 20px;
      border-radius: 8px;
      margin-bottom: 30px;
    }

    .control-group {
      margin-bottom: 20px;
    }

    .control-group h3 {
      margin-bottom: 10px;
      color: #333;
    }

    button {
      padding: 10px 20px;
      margin: 5px;
      border: none;
      border-radius: 5px;
      cursor: pointer;
      font-size: 14px;
      transition: transform 0.2s;
    }

    button:hover {
      transform: scale(1.05);
    }

    .demo-box {
      width: 300px;
      height: 300px;
      background-color: #007bff;
      color: white;
      display: flex;
      align-items: center;
      justify-content: center;
      font-size: 24px;
      border-radius: 8px;
      margin: 20px auto;
      transition: all 0.3s ease;
    }

    /* Classes de style */
    .rounded {
      border-radius: 50%;
    }

    .shadow {
      box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);
    }

    .bordered {
      border: 5px solid #ffd700;
    }

    .gradient {
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    }

    .animated {
      animation: pulse 2s infinite;
    }

    @keyframes pulse {
      0%,
      100% {
        transform: scale(1);
      }
      50% {
        transform: scale(1.05);
      }
    }

    /* Thème sombre */
    body.dark-theme {
      background-color: #1a1a1a;
      color: #ffffff;
    }

    body.dark-theme .controls {
      background-color: #2d2d2d;
    }

    body.dark-theme .control-group h3 {
      color: #ffffff;
    }

    body.dark-theme .card {
      background-color: #2d2d2d;
    }

    /* Galerie de cartes */
    .cards-container {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
      gap: 20px;
      margin-top: 30px;
    }

    .card {
      background-color: white;
      border-radius: 8px;
      padding: 20px;
      box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
      transition: all 0.3s ease;
    }

    .card.highlight {
      background-color: #fff3cd;
      border-left: 5px solid #ffc107;
    }

    .card.success {
      background-color: #d4edda;
      border-left: 5px solid #28a745;
    }

    .card.danger {
      background-color: #f8d7da;
      border-left: 5px solid #dc3545;
    }

    .card:hover {
      transform: translateY(-5px);
      box-shadow: 0 5px 20px rgba(0, 0, 0, 0.2);
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>Styliste Dynamique</h1>

    <div class="controls">
      <div class="control-group">
        <h3>1. Modifier la boîte avec classList</h3>
        <button onclick="toggleRounded()">Toggle Arrondi</button>
        <button onclick="toggleShadow()">Toggle Ombre</button>
        <button onclick="toggleBorder()">Toggle Bordure</button>
        <button onclick="toggleGradient()">Toggle Gradient</button>
        <button onclick="toggleAnimation()">Toggle Animation</button>
        <button onclick="resetBox()">Réinitialiser</button>
      </div>

      <div class="control-group">
        <h3>2. Modifier la boîte avec style</h3>
        <button onclick="changeColor('red')">Rouge</button>
        <button onclick="changeColor('green')">Vert</button>
        <button onclick="changeColor('blue')">Bleu</button>
        <button onclick="changeSize('small')">Petit</button>
        <button onclick="changeSize('medium')">Moyen</button>
        <button onclick="changeSize('large')">Grand</button>
      </div>

      <div class="control-group">
        <h3>3. Thème de la page</h3>
        <button onclick="toggleTheme()">Toggle Thème</button>
      </div>

      <div class="control-group">
        <h3>4. Galerie de cartes</h3>
        <button onclick="highlightCards('highlight')">Highlight</button>
        <button onclick="highlightCards('success')">Success</button>
        <button onclick="highlightCards('danger')">Danger</button>
        <button onclick="highlightCards('none')">Aucun</button>
      </div>
    </div>

    <div class="demo-box" id="demo-box">Boîte de démo</div>

    <div class="cards-container" id="cards-container">
      <div class="card">
        <h3>Carte 1</h3>
        <p>Contenu de la première carte</p>
      </div>
      <div class="card">
        <h3>Carte 2</h3>
        <p>Contenu de la deuxième carte</p>
      </div>
      <div class="card">
        <h3>Carte 3</h3>
        <p>Contenu de la troisième carte</p>
      </div>
      <div class="card">
        <h3>Carte 4</h3>
        <p>Contenu de la quatrième carte</p>
      </div>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```



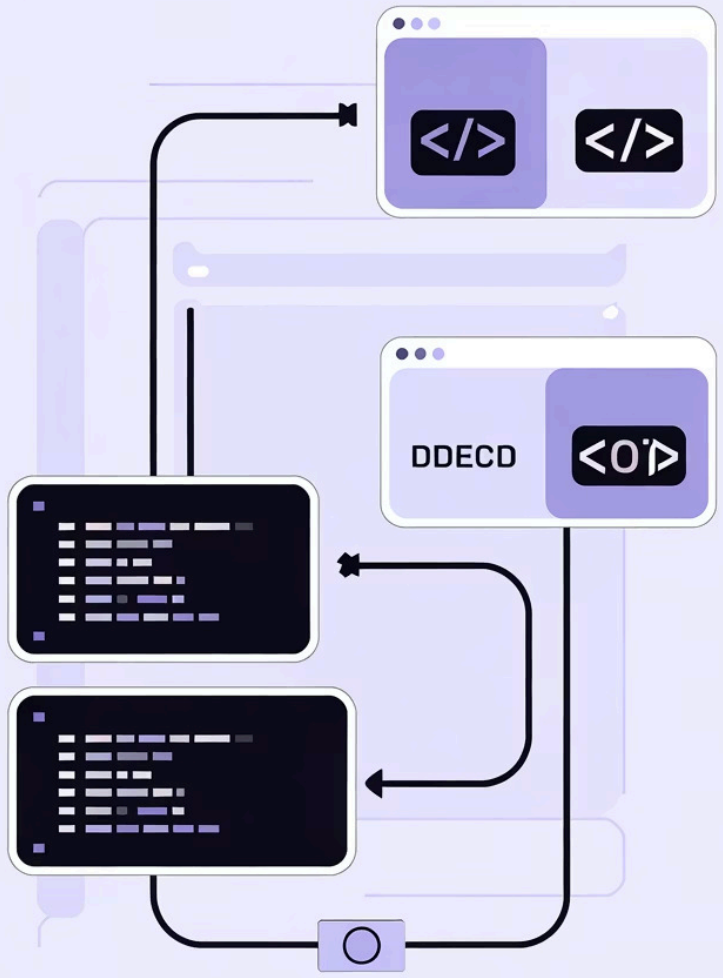
MODULE

Modifications des attributs

EXPLICATIONS

Interaction avec les attributs HTML

Les attributs HTML fournissent des informations supplémentaires sur les éléments. JavaScript permet de lire, modifier, ajouter et supprimer ces attributs dynamiquement.



EXPLICATIONS

getAttribute(name)

```
const link = document.querySelector("a");
const href = link.getAttribute("href");
console.log(href); // "#home"

const img = document.querySelector("img");
const src = img.getAttribute("src");
const alt = img.getAttribute("alt");
```

EXPLICATIONS

setAttribute(name, value)

```
const link = document.querySelector("a");
link.setAttribute("href", "https://example.com");
link.setAttribute("target", "_blank");
link.setAttribute("rel", "noopener noreferrer");

const img = document.querySelector("img");
img.setAttribute("src", "nouvelle-image.jpg");
img.setAttribute("alt", "Description de l'image");
```

EXPLICATIONS

removeAttribute(name)

```
const link = document.querySelector("a");
link.removeAttribute("target");

const input = document.querySelector("input");
input.removeAttribute("disabled");
```

EXPLICATIONS

hasAttribute(name)

Cette méthode booléenne permet de vérifier si un élément HTML possède un attribut spécifique. Elle est très utile pour ajouter de la logique conditionnelle basée sur la présence ou l'absence d'un attribut.

```
const link = document.querySelector("a");

if (link.hasAttribute("target")) {
  console.log("Le lien s'ouvre dans un nouvel onglet");
}
```

EXPLICATIONS

Attributs data-*

Les attributs `data-*` permettent de stocker des données personnalisées sur les éléments HTML. Ils sont accessibles via la propriété `dataset`.

```
<div class="product"
    data-id="123"
    data-price="29.99"
    data-category="electronics">
   Produit
</div>
```

EXPLICATIONS

Utilisation des attributs data-* avec JavaScript

```
const product = document.querySelector(".product");

// Accès via dataset (sans le préfixe data-)
console.log(product.dataset.id);       // "123"
console.log(product.dataset.price);    // "29.99"
console.log(product.dataset.category); // "electronics"

// Modification
product.dataset.stock = "15";
product.dataset.featured = "true";

// Les attributs multi-mots utilisent camelCase
// data-product-name → dataset.productName
product.dataset.productName = "Ordinateur Portable";

// Vérification
if (product.dataset.featured === "true") {
  console.log("Produit en vedette");
}
```

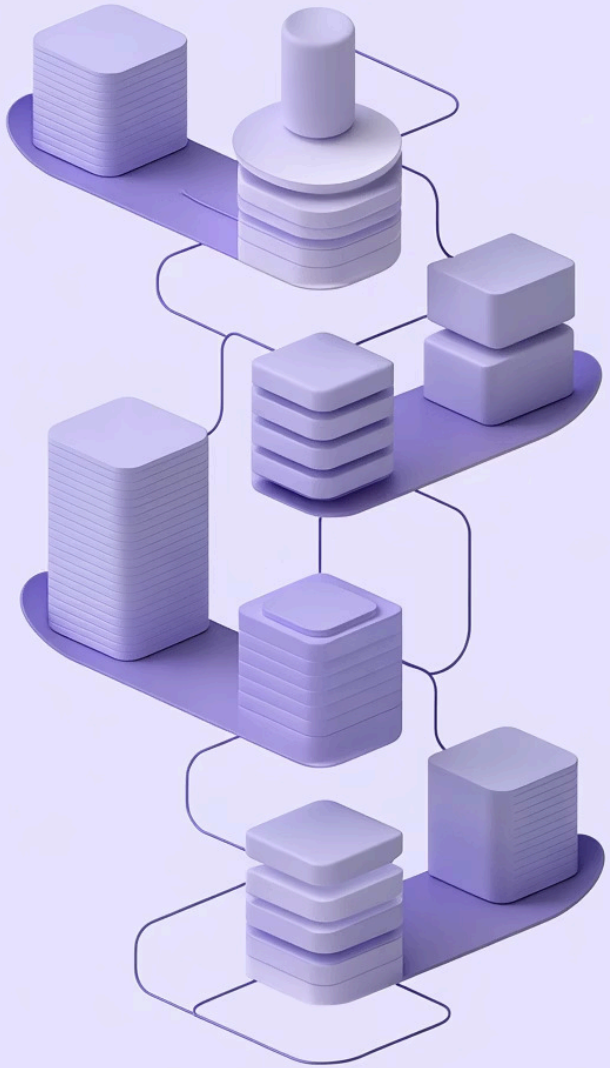
MISE EN PRATIQUE

Construire votre portfolio avec JavaScript

Le moment est venu de mettre en œuvre ce que nous avons appris ! Votre objectif est de créer une page web de portfolio, entièrement construite à l'aide de JavaScript.

- Le fichier HTML de base doit être minimal : uniquement la structure de base (`<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`) et les liens vers vos fichiers CSS et JavaScript.
- Tout le contenu visible (éléments de navigation, sections, images, texte) doit être généré et inséré dans le DOM par JavaScript.

Bonus : Au lieu d'utiliser `innerHTML`, explorez des alternatives comme `document.createElement()` et `appendChild()` pour une meilleure performance et sécurité.



MODULE

Insertion et suppression d'éléments

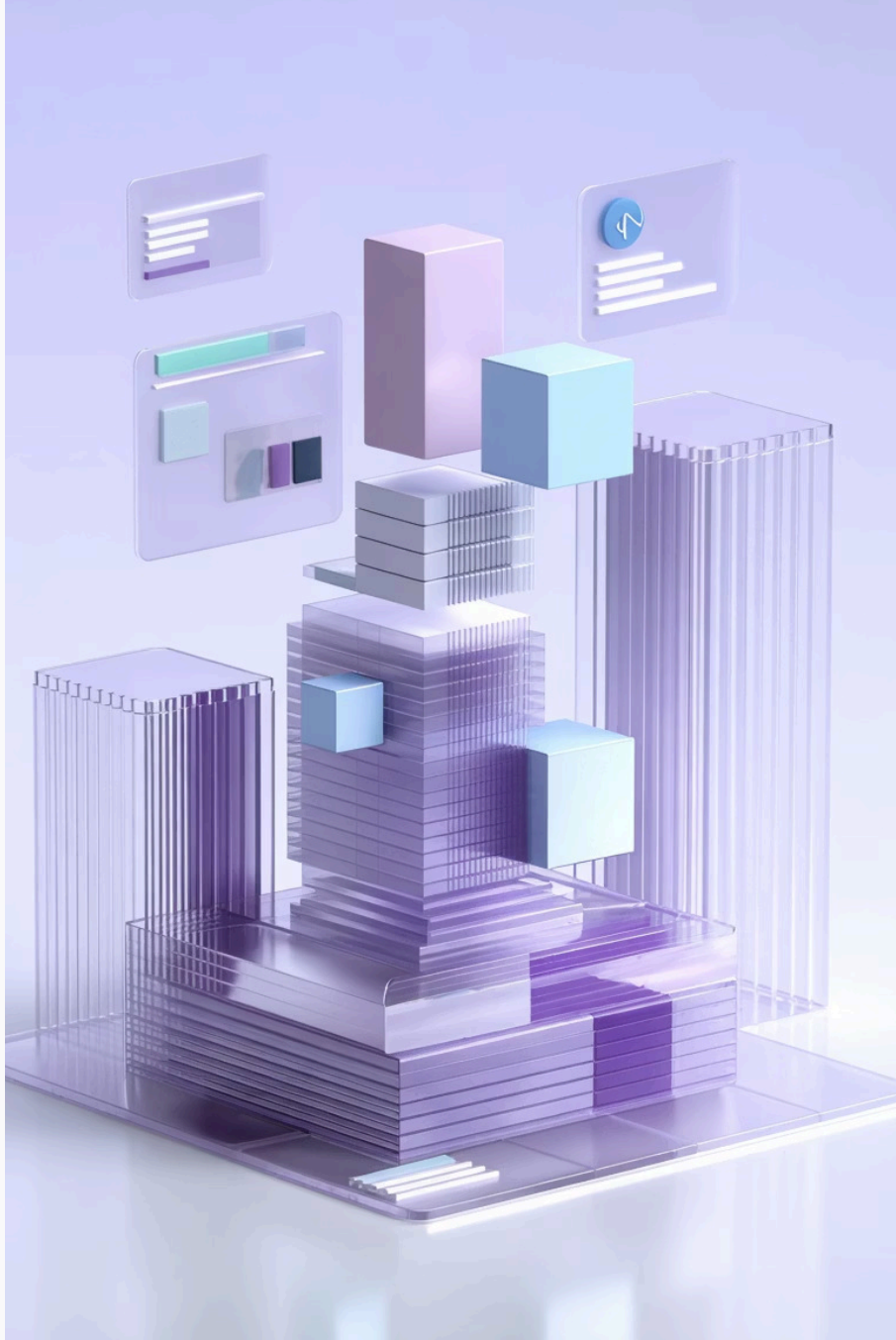
EXPLICATIONS

innerHTML est mort, vive createElement !

Alors que `innerHTML` est pratique pour insérer rapidement du contenu HTML, il présente des risques de sécurité (XSS) et peut être moins performant.

La méthode `document.createElement()` est la solution à privilégier pour créer de nouveaux nœuds d'éléments HTML.

```
// Création de divers éléments HTML
const div = document.createElement('div');
const paragraph = document.createElement('p');
const button = document.createElement('button');
const link = document.createElement('a');
```



EXPLICATIONS

Méthodes d'insertion d'éléments

Pour enrichir le DOM, JavaScript offre plusieurs méthodes pour insérer de nouveaux éléments HTML à des positions spécifiques. Nous allons explorer `append()`, `prepend()` et `insertBefore()`.

EXPLICATIONS

appendChild(element)

La méthode `appendChild()` est utilisée pour insérer un nœud comme dernier enfant d'un nœud parent spécifié. Elle est très utile pour ajouter de nouveaux éléments à la fin d'un conteneur existant.

```
const container = document.getElementById('container');
const newParagraph = document.createElement('p');
newParagraph.textContent = "Nouveau paragraphe ajouté avec appendChild !";

container.appendChild(newParagraph);
```

EXPLICATIONS

append(...elements)

La méthode moderne `append()` permet d'insérer plusieurs nœuds (éléments ou chaînes de texte) à la fin du nœud parent spécifié. Contrairement à `appendChild()`, elle accepte plusieurs arguments et peut insérer du texte brut directement.

```
const container = document.getElementById('container');

const title = document.createElement('h2');
title.textContent = "Titre de section";

const text = document.createElement('p');
text.textContent = "Voici un paragraphe de texte ajouté dynamiquement.";

// Ajouter plusieurs éléments (y compris du texte) en une seule fois
container.append(title, text, "Et même une simple chaîne de texte !");
```

EXPLICATIONS

prepend(...elements)

La méthode `prepend()` insère un ou plusieurs nœuds (éléments ou chaînes de texte) au début du nœud parent spécifié. C'est l'inverse de `append()` et est très utile pour ajouter de nouveaux éléments en tête d'un conteneur.

```
const list = document.getElementById('my-list');
const firstItem = document.createElement('li');
firstItem.textContent = "Premier élément";

list.prepend(firstItem); // Ajoute au début
```

EXPLICATIONS

insertBefore(newElement, referenceElement)

La méthode `insertBefore()` insère un nœud (`newElement`) juste avant un nœud enfant existant (`referenceElement`) d'un nœud parent spécifié. Elle est essentielle pour un contrôle précis de la position des éléments dans le DOM.

```
const container = document.getElementById('container');
const existingElement = document.getElementById('existing'); // L'élément devant lequel insérer

const newElement = document.createElement('div');
newElement.textContent = "Je suis inséré avant l'élément existant";

container.insertBefore(newElement, existingElement);
// Si existingElement est null, newElement est ajouté à la fin, comme avec appendChild.
```

MISE EN PRATIQUE

Mise en pratique : Remplacer innerHTML

Retournons à la création de votre portfolio. Cette fois, l'objectif est de refactoriser le code existant en remplaçant toutes les utilisations de `innerHTML` par des méthodes de création et d'insertion d'éléments plus sûres et plus performantes en JavaScript.

EXPLICATIONS

remove()

La méthode `remove()` est la façon la plus simple et moderne de supprimer un élément du DOM. Elle est appelée directement sur l'élément que vous souhaitez faire disparaître, sans avoir besoin de faire référence à son parent.

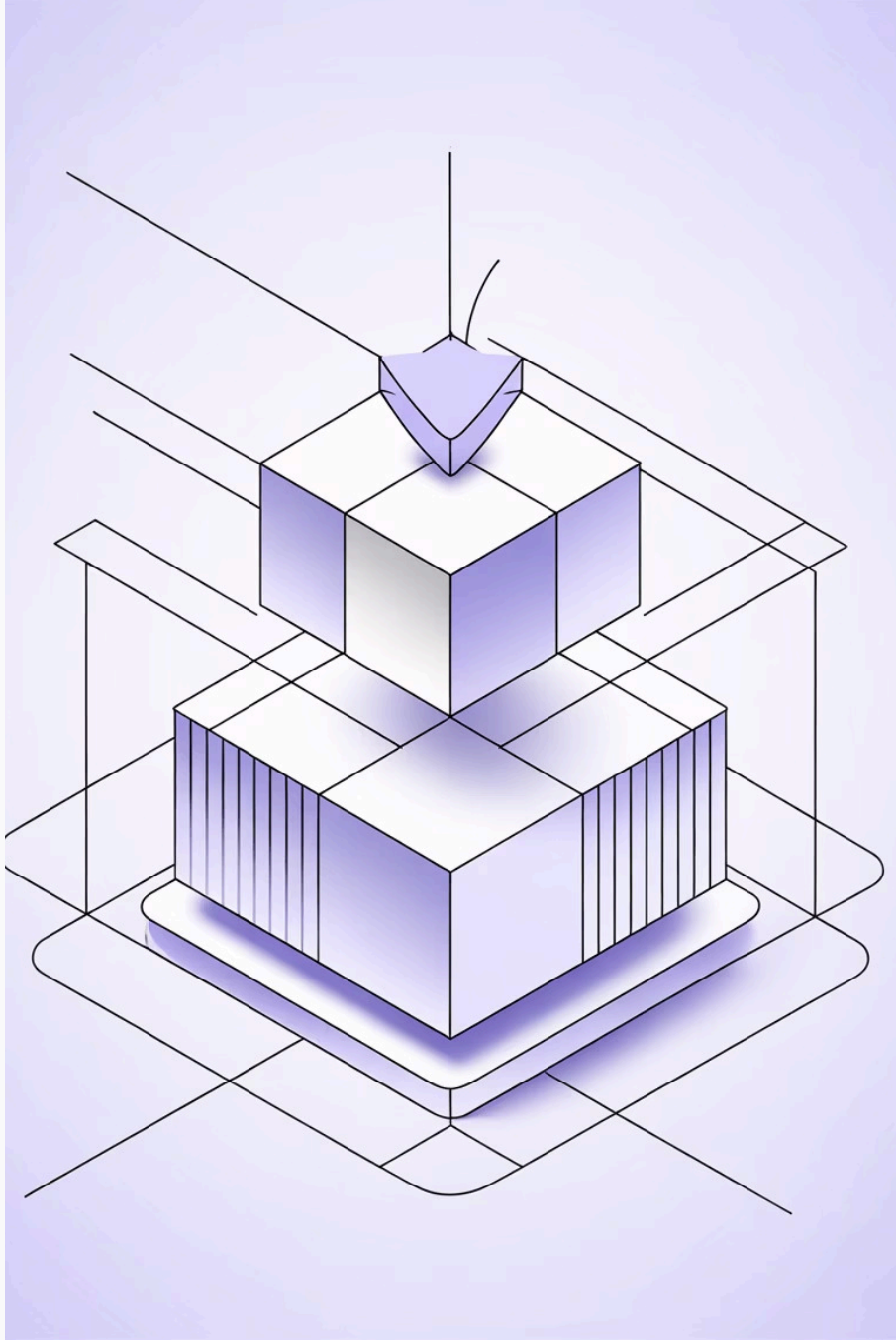
```
const element = document.getElementById('to-remove');
element.remove(); // L'élément est retiré du document
```

EXPLICATIONS

removeChild(child)

La méthode `removeChild()` est une méthode plus ancienne mais toujours valide pour supprimer un nœud enfant spécifié d'un nœud parent. Contrairement à `remove()`, elle doit être appelée sur le nœud parent et prend en argument le nœud enfant à supprimer.

```
const container = document.getElementById('container');
const child = document.getElementById('child-to-remove');

// Supprime l'élément enfant 'child' du 'container' parent
container.removeChild(child);
```



EXPLICATIONS

Supprimer tous les enfants d'un élément

Il est souvent nécessaire de vider le contenu d'un élément parent avant d'y ajouter de nouveaux éléments, par exemple lors de la mise à jour d'une liste ou d'une section. JavaScript offre plusieurs méthodes pour y parvenir, chacune avec ses avantages et inconvénients.

```
const container = document.getElementById('container');

// Méthode 1 : innerHTML
container.innerHTML = '';

// Méthode 2 : Boucle
while (container.firstChild) {
    container.removeChild(container.firstChild);
}

// Méthode 3 : replaceChildren (moderne)
container.replaceChildren();
```

CONCLUSION

Et… on s'arrête là pour les notions du jour !

Ça fait beaucoup (vraiment), donc on fait une pause pour aujourd'hui.

… mais ce n'est pas tout à fait fini 😏

MISE EN PRATIQUE

Galerie photo interactive

Contraintes

- Permettre d'afficher les photos de toutes les catégories ou de la catégorie active sélectionnée.
- Chaque catégorie doit contenir au moins deux photos distinctes.
- À chaque changement de catégorie, vous devrez supprimer l'intégralité de la liste des photos affichées pour ensuite la reconstruire dynamiquement avec les photos pertinentes.

Structure d'une image (Objet JavaScript)

Chaque photo sera représentée par un objet JavaScript avec la structure suivante :

```
{
  id: 1,
  titre: "Nom de la photo",
  description: "Courte description de l'image.",
  thumbnail: "URL_de_la_miniature.jpg", // Petite image pour l'affichage de la galerie
  imageLarge: "URL_de_l'image_pleine_taille.jpg", // Grande image pour le bonus modal
  categorie: "Nature" // Exemple : "Nature", "Villes", "Animaux"
}
```

MISE EN PRATIQUE

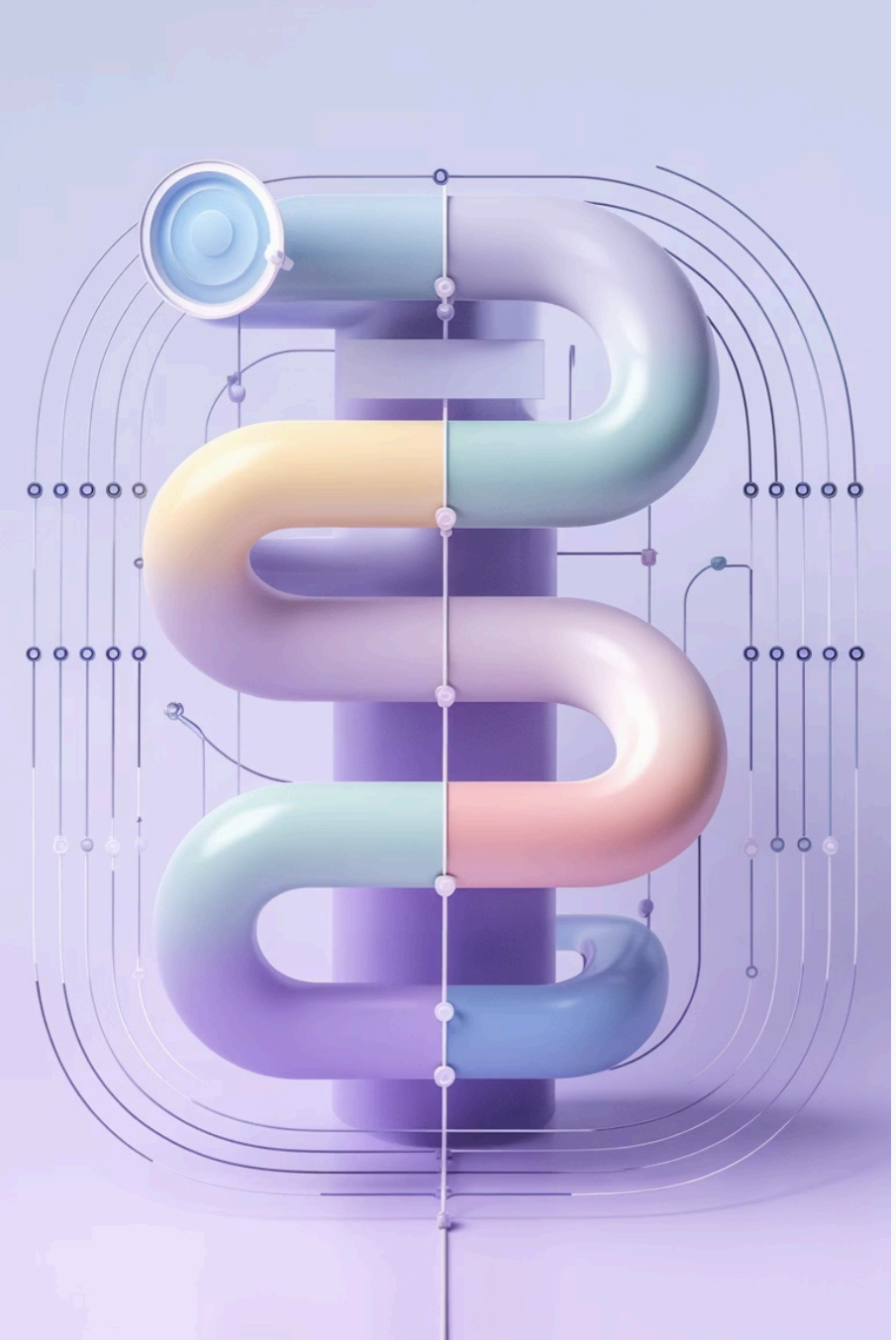
Galerie photo interactive

Conseils

- Mettez en place un objet pour stocker les informations de l'application, notamment pour stocker la catégorie active. Ça vous permettra de tester sans mettre en place d'interactions que la génération de la liste de photos est fonctionnelle !
- Pour ne pas perdre de temps à trouver des images, utilisez Lorem Picsum (**https://picsum.photos**). Vous pouvez spécifier des dimensions et obtenir des images aléatoires.
- Pour la gestion des interactions (par exemple, le clic sur un bouton de catégorie), inspirez-vous de la manière dont `onclick` a été utilisé lors des précédentes mises en pratique pour les styles.

Bonus : Fenêtre Modale

Pour aller plus loin, implémentez un bonus : au clic sur une miniature de photo dans la galerie, affichez l'image large correspondante avec sa description dans une fenêtre modale qui recouvre le reste du contenu.



À SUIVRE

Prochaines étapes

Maintenant que nous avons les bases de la manipulation du DOM, on va pouvoir démarrer tranquillement les réelles interactions avec les événements.

Et comme d'habitude, ce qu'on a vu jusqu'à aujourd'hui sera approfondi au fur et à mesure !